第 2-2 讲 • 传菜口：preload 与 IPC

> **模块 2 • 认清记账簿的骨架** ｜ 建议用时 **40 分钟**（阅读 22 + 动手 18）
> 前置要求：完成 2-1（理解主进程 / 渲染进程的分工）

本节目标

读完并做完这一讲，你能：

1. 说出**一条请求要经过的四个地方**，并按顺序背出来。
2. 解释 `preload` 为什么必须是"唯一的通道"，以及它比"直接开放权限"好在哪里。
3. 在项目里**独立找到**任意一条链路的四个环节（这是读代码的核心技能）。
4. 理解 `invoke` 与"等待结果"的机制：为什么界面按一下要等一下才有反应。
5. 能解释一个真实场景：**点了保存，数据是怎么一步步落到硬盘上的。**

一、一条请求的旅程

1.1 现实比喻：在餐厅点菜

客人（用户）想吃饭，流程是：

①	客人看菜单，决定要宫保鸡丁	→ 界面上的按钮
②	服务员写下"宫保鸡丁"这张单子	→ `preload` 把请求翻译成正式格式
③	单子通过传菜口递进后厨	→ `IPC` 通道
④	后厨真的开火做菜	→ 主进程执行（写数据库）
⑤	菜做好，从传菜口递回来	→ 返回值
⑥	服务员端到客人面前	→ 界面显示结果

关键规则：客人**不能自己冲进后厨**，服务员**不能自己做菜。必须走这条路。**

1.2 对应到代码：四个地方

记账簿里"保存一笔账"这条链路，**要在四个地方各写一段**：

环节	在哪个文件	干什么	真实代码（节选）
①名字	`src\shared\ipc.ts`	给这条请求起个唯一名字	`recordsCreate: 'records:create'`
②通道	`src\preload\index.ts`	暴露一个函数给界面用	`create: (input) => ipcRenderer.invoke(IPC.recordsCreate, input)`
③执行	`src\main\ipc\records.ts`	真正干活：写数据库	`ipcMain.handle(IPC.recordsCreate, (_e, input) => {...})`
④调用	界面组件（如 `RecordForm.tsx`）	用户点按钮时发起请求	`await window.api.records.create(payload)`

记住这个顺序：**名字 → 通道 → 执行 → 调用。**

这条链路是理解整个项目的钥匙。 以后你要找"某个功能在哪"，就顺着这四个环节找；要加一个新功能，就是把这四段各写一遍。

二、为什么要有 preload 这一层

2.1 天真的做法（以及为什么不行）

你可能会想：既然界面要请求后厨，直接让界面能调 `ipcRenderer` 不就行了？为什么要多一层 preload？

因为那样等于把后厨的钥匙给了客人。

Electron 里有一个危险的配置：`nodeIntegration: true`。开了它，界面代码就能直接写：

```
// 如果开了 nodeIntegration，界面里就能这么干：
const fs = require('fs')
fs.unlinkSync('C:\\你的数据目录\\daily-ledger.db')   // 删掉账本
```

一行代码，账没了。

2.2 preload 的做法：只开一扇窗，窗上装栅栏

有了 preload，界面能做到的事**被严格限定成一份清单**：

```
// src\preload\index.ts
contextBridge.exposeInMainWorld('api', {
  records: {
    list: (query) => ipcRenderer.invoke(IPC.recordsList, query),
    create: (input) => ipcRenderer.invoke(IPC.recordsCreate, input)
  },
  accounts: { ... },
  budgets: { ... }
})
```

这段代码在说：

> "界面里会多出一个叫 `window.api` 的东西。它**只有这些方法**，一个不多。
> 每个方法能做的事，就是我在这里写死的那一件。**界面无法自己发明新请求。**"

于是界面的能力清单变成了可审计的一页纸：它能查账、能记账、能改设置——但**它没有"删除数据库文件"这种方法**，因为它压根调不到。

2.3 三层防护，一层都不能少

层	机制	挡住什么
1	`nodeIntegration: false`	界面**不能**直接使用系统能力（第 2-1 讲）
2	`contextIsolation: true`	界面**看不见** preload 内部的变量
3	`contextBridge` + 白名单	界面**只能用**清单上列出的方法

三层叠加才是完整的。只有第 1 层，界面还能想办法碰到 preload 的东西；只有第 3 层，可能从别的地方漏。

这就是"纵深防御"：不指望一道墙拦住所有情况，而是让攻击者要连破三道。

三、`invoke`：为什么要"等"

3.1 菜要现做

`ipcRenderer.invoke(...)` 这个 `invoke` 的意思是"**请求并等待结果**"。

```
// 界面的写法
const saved = await window.api.records.create(payload)
//            ↑ 这个 await 就是"等后厨把菜做完"
```

为什么必须等：因为后厨做事要时间——写数据库、算余额、更新账户，都是真实操作。**界面不能假装已经完成**（那会在出错的场景下显示错误结果）。

3.2 生活比喻：点外卖

下单之后，你会**等**骑手送达，而不是立刻跟朋友说"我吃完了"。

`await` 就是"等送达"：

```
// 不等（错误做法）
const result = window.api.records.create(payload)   // result 是一个"还没送到"的承诺
console.log(result.id)                              // ❌ 报错：东西还没到

// 等（正确做法）
const result = await window.api.records.create(payload)
console.log(result.id)                              // ✅ 已经存好了，拿到真实 id
```

这就是为什么有时候界面会"转圈"——它在等后厨做完。

3.3 出错了怎么办

等待还有一个好处：**能接住错误。**

```
try {
  await window.api.records.create(payload)
  toast('已保存', 'ok')
} catch (error) {
  toastError(error)     // 后厨说"做不了"，服务员告诉客人
}
```

如果后厨拒绝（比如金额为 0、日期非法），这个错误会沿着原路返回，界面就能弹出提示。**这是"错误处理"的基础**，第 4-4 讲会专门讲怎么读懂这些错误。

四、动手任务

步骤 1：完整追一条链路（本讲核心）

我们要追的是「**界面查询账目列表**」这条链路。**四个地方，逐个找。**

① 名字：

```
cd "C:\Users\34123\Desktop\记账簿"
Select-String -Path src\shared\ipc.ts -Pattern "recordsList"
```

预期看到：`recordsList: 'records:list',`

记住这个名字：**`records:list`**。它是这条链路的"身份证"。

② 通道：

```
Select-String -Path src\preload\index.ts -Pattern "recordsList"
```

预期看到：`list: (query) => ipcRenderer.invoke(IPC.recordsList, query),`

注意：这里用的是 `IPC.recordsList`（引用常量），不是硬写字符串 `'records:list'`。**为什么要用常量**：如果哪天要改名，只改 `ipc.ts` 一处，四个地方自动同步——**改字符串就得四处找，必然漏一处。**

③ 执行：

```
Select-String -Path src\main\ipc\records.ts -Pattern "recordsList" -Context 0,6
```

预期看到 `ipcMain.handle(IPC.recordsList, ...)` 以及它后面几行。

这就是后厨真正干活的地方——它会去问数据库，然后把结果返回。

④ 调用：

```
Get-ChildItem src\renderer\src -Recurse -Include *.tsx,*.ts | Select-String "window.api.re
```

预期看到界面里发起请求的地方。

恭喜：你刚刚独立走通了一条完整的 IPC 链路。**这就是读这个项目代码的基本功。**

步骤 2：验证"白名单"的存在

```
Get-Content src\preload\index.ts | Select-String "exposeInMainWorld"
```

预期看到：`contextBridge.exposeInMainWorld('api', {` 。

现在数一数这个清单里有多少个顶层分类：

```
Get-Content src\preload\index.ts | Select-String "^  \w+: \{"
```

预期看到：`app:`、`settings:`、`accounts:`、`categories:`、`records:`、`attachments:`、`calendar:`、`recurring:`、`billShot:`、`exchange:`、`stats:`、`budgets:`、`security:`

实测数字：**13 个分类，一共 60 个方法。**

这就是界面能做的全部事情的清单，一页纸就能看完。

思考：这个清单里有"删除数据库文件"吗？

答案：**没有。**而且界面**无法自己加**——因为它改不了 preload 的代码（那是主进程侧的文件）。**这就是白名单的威力。**

可以自己验证一遍（这个检查很有说服力）：

```
Get-Content src\preload\index.ts | Select-String "unlink|rmdir|exec|spawn|rmSync|eval"
```

预期：没有任何输出。 也就是说，暴露给界面的 60 个方法里，**没有一个能直接删文件、执行系统命令或动态执行代码。**界面只能"请求后厨做事"，不能自己动手。

步骤 3：体验"命名约定"

```
Select-String -Path src\shared\ipc.ts -Pattern "':" | Select-Object -First 12
```

预期看到类似：

```
recordsList: 'records:list',
recordsCreate: 'records:create',
accountsList: 'accounts:list',
budgetsCreate: 'budgets:create',
```

规律：名字都是 `模块:动作` 的格式（`records:create`、`budgets:list`、`app:choose-data-dir`）。

为什么要有约定：这个项目有 **60 个通道**，没有规律就乱成一团。有约定之后：

- 看到 `billshot:preview` 就知道是"账单截图模块的预览动作"
- 报错里出现 `Error invoking remote method 'records:create'`，你立刻知道该去 `main\ipc\records.ts` 找

这是"好代码"的一个特征：命名本身就携带信息。

自己数一数：

```
(Select-String -Path src\shared\ipc.ts -Pattern "^\s+\w+:\s*'[a-z-]+:" | Measure-Object).C
```

预期输出 60 —— 你和这个软件的每一次交互，都走这 60 条通道中的一条。

步骤 4：自己找一条陌生的链路

不看讲义，自己找出「**预算列表**」（`budgets:list`）的四个环节：

```
# ① 名字
Select-String -Path src\shared\ipc.ts -Pattern "budgetsList"
# ② 通道
Select-String -Path src\preload\index.ts -Pattern "budgetsList"
# ③ 执行
Select-String -Path src\main\ipc\budgets.ts -Pattern "budgetsList"
# ④ 调用
Get-ChildItem src\renderer\src -Recurse -Include *.tsx | Select-String "budgets.list"
```

如果四步都找到了，说明你已经掌握了这个项目最核心的结构。

卡住了怎么办

现象	原因	怎么办
搜不到 `recordsList`	名字可能有差异	先 `Select-String src\shared\ipc.ts -Pattern "records"` 看全部
界面里搜不到调用	可能封装在 store 里了	先搜 `window.api.records`，再顺着找
不理解 `_event` 参数	那是 Electron 传进来的事件对象，本项目用不到所以加下划线	下划线是"我知道它存在但不用"的约定
看到 `ipcMain.handle` 一脸懵	它就是"注册一个处理函数"	类比：告诉传菜口"这类单子以后交给我"

五、自测题

1. 一条 IPC 请求要经过哪四个地方？按顺序说出来。

2. 为什么不让界面直接调用 `ipcRenderer`，非要经过 preload？

3. `contextBridge.exposeInMainWorld` 做的事，用一句话概括是什么？

4. 为什么 IPC 名字都写成 `模块:动作` 的格式？

5. 界面调用时为什么要写 `await`？不写会怎样？

答案与解析

1. ① **名字**（`shared\ipc.ts` 定义）→ ② **通道**（`preload\index.ts` 暴露给界面）→ ③ **执行**（`main\ipc\*.ts` 里真正干活）→ ④ **调用**（界面组件发起请求）。 → 这条链路要背下来，它是你以后找代码、加功能的路线图。

2. 因为那样等于**把后厨钥匙给了客人**：界面代码一旦能直接用系统能力，一段恶意脚本就能删掉数据库。 → preload 把界面能做的事**限定成一份白名单**，一页纸就能审计完。

3. "在界面的 `window` 上挂一个叫 `api` 的对象，里面的方法**仅限我列出的这些**。" → 关键词是**仅限**——界面无法发明新请求。

4. 因为有 **100 多个通道**，必须有规律才能管理。 → 好处：看到名字就知道属于哪个模块、干什么；报错里出现 `records:create` 你能立刻定位到 `main\ipc\records.ts`。

5. 因为后厨做事需要时间（写数据库、算余额）。 `await` 是"**等结果回来再继续**"。 → 不写的话，你拿到的是一个"还没送达的承诺"，直接用里面的字段会报错；而且**错误也接不住**（没法用 `try/catch`）。

六、术语表

术语	一句话解释
IPC	进程间通信，主进程与渲染进程之间传消息的正式途径
preload	运行在中间的脚本，负责把"安全的方法"暴露给界面
`contextBridge`	Electron 提供的桥，用来安全地往界面挂方法
`exposeInMainWorld`	把对象挂到界面的 `window` 上（如 `window.api`）
白名单	只允许清单上的操作，其它一律拒绝
`ipcMain.handle`	主进程侧"注册一个处理函数"，接住某个通道的请求
`ipcRenderer.invoke`	渲染进程侧"发起请求并等待结果"
`await`	等待异步操作完成再继续
异步 / Async	需要时间才能完成的操作（读文件、查数据库）
异常 / Exception	执行过程中出错，可以被 `try/catch` 接住
纵深防御	不靠单一防线，而是多层叠加提高安全性
契约 / Contract	两边约定好的接口形状：叫什么、收什么、返回什么

七、本讲与项目的对应

- 你追踪的链路：`records:list`、`budgets:list`（四个环节全部找到）
- 你查看的文件：`src\shared\ipc.ts`、`src\preload\index.ts`、`src\main\ipc\records.ts`、界面组件
- **本讲不修改任何代码，不影响你的记账数据**
- 上一讲：2-1 主进程与渲染进程 ｜ 下一讲：**2-3 冷库：数据库与迁移**（数据到底怎么存的）

讲义版本：v1 ｜ 对应项目版本：0.1.0